What Are Developers Actually Discussing When Visual Regression Tests Fail?

Miku Watanabe*, Kosei Horikawa*, Brittany Reid*, Yutaro Kashiwa*, Hajimu Iida*
*Nara Institute of Science and Technology, Japan

***Abstract*—Visual Regression Tests (VRTs) are widely adopted as a mechanism for detecting unintended visual changes in user interfaces. By design, VRTs operate on rendered pixel output, and the prevailing assumption is that they catch stylistic regressions such as layout shifts, color mismatches, and font alterations. We conduct an empirical analysis of 307 pull requests (PRs) from 103 GitHub repositories that incorporate VRT results via Chromatic, comparing them against 299 PRs that contain image attachments but no VRT (Visual PRs). Quantitatively, VRT-PRs show no significant acceptance-rate difference, but exhibit a 3.8 times longer median resolution time, 10 times more discussion comments, and 1.75 to 4.5 times larger code changes than Visual PRs. VRT results are typically shared around the midpoint of the review process, sustaining ongoing discussion rather than serving only as a final check. Through a card-sorting analysis of 189 VRT-flagged issues, we identify seven defect categories assigned to the analyzed issues: Layout (39.7%), Appearance (27.5%), Color (14.8%), Text (9.5%), State (6.9%), Test (6.3%), and Image (4.2%). The three most frequent categories are stylistic, while approximately 18.5% of analyzed issues (35/189) involve non-stylistic origins, including undefined component state (13 cases), content disappearance (17 cases across multiple categories), and visually imperceptible regressions (5 cases). We further document cases in which VRT detected visual regressions originating from code changes in seemingly unrelated files, exposing non-local effects that no targeted test would have been written to catch. These observations indicate that, in addition to its primary role as a stylistic checker, VRT functions as a secondary detector of unintended consequences of code changes, with implications for how VRT should be integrated into the maintenance toolchain.**

***Index Terms*—Visual Regression Tests, User Interface, PRs**

I. Introduction

The quality of a user interface significantly impacts user ratings [1], motivating developers and companies to invest substantial effort in UI improvement [2]. UI quality has measurable effects on user satisfaction [3][4], engagement [5], and product success [6], making it a critical component of modern software engineering practice.

Despite this importance, user interfaces are inherently fragile [7]. Minor changes in layout, styling, or component behavior can inadvertently degrade user experience [8]. This fragility intensifies in large-scale applications where multiple developers work on the same codebase, often introducing inconsistencies or regressions [9]. As applications evolve through frequent updates and feature additions, maintaining a consistent, high-quality interface becomes increasingly difficult [10].

To address this challenge, many development teams have adopted Visual Regression Tests (VRTs). VRTs automatically capture and compare UI screenshots before and after code changes to detect unintended visual differences. By integrating VRT into the development workflow, teams can identify UI anomalies early in the development cycle, reducing the risk of deploying broken or inconsistent interfaces. Technical blogs report that this proactive approach safeguards user experience while enhancing development efficiency through immediate feedback on visual changes [11].

Despite VRT's widespread adoption, only a few studies have examined these tests [12][13]. These studies proposed approaches to detect UI inconsistencies and enhance visual regression testing. To the best of our knowledge, no empirical studies have analyzed VRT activities in practice. Consequently, how visual regression tests contribute to front-end development and what challenges developers encounter when using VRTs remain unclear.

This study clarifies how visual regression tests impact software development and identifies issues detectable through VRTs. We collected the outcomes of VRTs from GitHub by identifying Pull Requests (PRs) linked to Chromatic [14], a UI quality management service. Chromatic runs VRT on Storybook [15], a widely used UI component development environment, and visualizes UI changes via a web application. Analyzing VRT results on Chromatic enables us to examine UI improvement activities in real development contexts.

Our empirical analysis of 307 PRs using Chromatic shows that they are associated with more active discussions and longer merge times. Furthermore, these PRs show a median number of changed files 1.75 times greater than those without VRTs. Additionally, our manual inspection revealed that VRTs not only detect visual changes but also facilitate the identification of unexpected functional regressions.

Replication Package: To facilitate replication studies and future extensions, the data used in our work is publicly available in the replication package [16].

II. Background

A. Visual Regression Tests

Visual Regression Testing (VRT) is a form of software testing that detects unintended visual changes in a user interface (UI) across updates. It ensures the visual presentation of a web or mobile application stays consistent after code changes. A related technique, GUI Testing, simulates user interactions by locating and manipulating UI elements to validate functional behavior, whereas VRT compares screenshots taken before and after changes.

VRT renders the UI in a controlled environment, typically a headless browser, and captures baseline screenshots

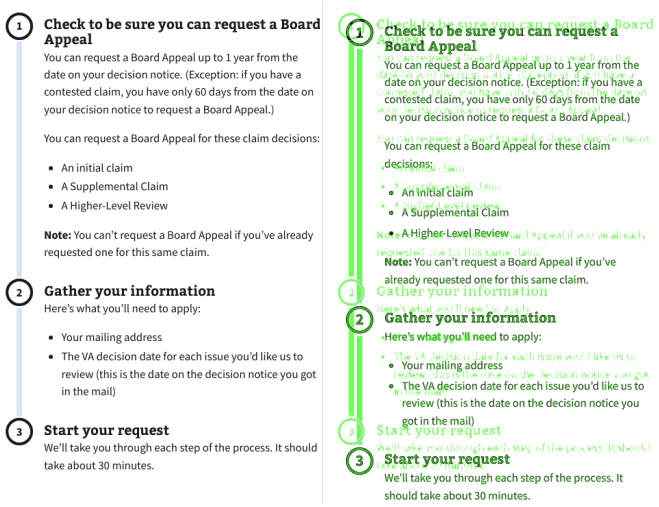

Fig. 1: Example of UI Regressions Detected by VRT [18]

of the expected state. After code changes, it captures new screenshots of the updated UI and compares them against the baseline using per-pixel or perceptual diffing algorithms. When discrepancies are found, the tool produces a diff image highlighting them. As shown in Fig. 1, VRT tools compare a baseline snapshot (left) with an updated snapshot (right) and flag anomalies such as layout shifts, color mismatches, or text misalignments, commonly highlighted in green.

Developers then review the differences to decide whether they are intentional design changes or regressions. Expected changes are approved and stored as the new baseline; otherwise, the issues are fixed before merging. For example, in one pull request [17], Chromatic flagged a dark mode rendering issue in the sub-navigation component of PictureLayout caused by a copy-paste error that had gone unnoticed, illustrating how VRT can catch subtle defects that might otherwise reach production.

VRT can be plugged into a continuous integration pipeline, enabling automated visual verification of every code change. Despite its growing industry adoption, little is known about the types of visual issues VRT detects, its coverage, and the operational challenges it introduces. This study empirically investigates the impacts of VRT on modern development practices.

B. Related Work

GUI testing verifies that the user interface behaves correctly from the user's perspective [19][20][21]. Although it can cost more upfront than manual testing, it tends to be more efficient overall, with higher defect detection rates [22]. Alégroth *et al.* [22] studied GUI Testing in two safety-critical software companies and found 9 defects, including 5 that manual testing had missed.

Several studies examine snapshot testing [23][24][25], a related concept. Both snapshot and visual regression tests detect unintended UI changes, but snapshot tests target a component's structural output (e.g., generated HTML or JSX) and cannot catch visual issues such as layout shifts or styling errors. Fujita *et al.* [24] found that projects using snapshot testing have over 1.6 times as many test cases as those that do not.

Other work has examined UI defects from several angles, including defect classification [26], defect location and impact [27], and user feedback [3]. Most closely related to our work, Kuramoto *et al.* [28] studied visual issue reports with images and videos on GitHub, finding that image-containing issues had fewer words than text-only reports while videos showed no word-count difference, and that visual information did not significantly increase discussion activity or speed up resolution.

While these studies characterize GUI functional testing, snapshot testing, or visual issue reports, none has empirically examined VRT activity in PR review contexts; we address this gap by analyzing VRT-linked PRs in their natural development setting.

III. Data Collection

We collected PRs from open-source projects that utilize Chromatic, a cloud-based VRT and UI review platform. Chromatic enables developers to detect visual and functional discrepancies in web user interfaces by comparing UI snapshots before and after code changes.

We focused on identifying PRs that include visual regression test results. Because many discussions occur in GitHub PR threads, we targeted PRs referencing Chromatic results to examine review discussions and outcomes in context.

To identify such PRs, we searched for those containing URLs linking to Chromatic test result pages, which typically begin with "`https://www.chromatic.com/test?`". We performed this search using the GitHub GraphQL API [29], configuring the query to exclude bot-generated comments. Our dataset includes PRs created between July 2, 2018 (the release date of Chromatic), and September 30, 2025.

This search yielded 314 PRs. We excluded 7 PRs that remained open (*i.e.,* still under development) and removed comments generated by bots that included VRT results. After filtering, we obtained 307 VRT-PRs across 103 repositories, with 373 comments containing Chromatic links.

We built a comparison dataset of PRs containing image attachments in their descriptions or comments, called Visual PRs [28]. Using the same 103 repositories, we searched for `<img>` HTML tags in PR descriptions or comments during the same period (July 2, 2018, to September 30, 2025). To ensure comparable representation, we applied stratified random sampling: for each repository with VRT-PRs, we randomly sampled an equal number of Visual PRs from the same period, excluding any VRT-PRs. This yielded 299 Visual PRs compared to 307 VRT-PRs. The difference reflects the limited availability of Visual PRs in these repositories.

IV. RESULTS

RQ_1: Do VRT-related Reviews Get Merged More and Faster?

Motivation. The outputs of VRTs provide concrete evidence of UI changes, which could either facilitate quicker approvals through clearer communication or extend review time by revealing issues requiring discussion. Kuramoto *et al.* [30] found that issues highlighted by images tend to be resolved more quickly. Since VRT provides more structured visual feedback than screenshots, it may similarly affect review resolution.

Approach. To investigate whether VRT presence is associated with PR acceptance and resolution time, we compare VRT-PRs and Visual PRs in terms of their acceptance rates and merge times. We define the acceptance rate as the proportion of merged PRs among all considered PRs (*i.e.,* both merged and closed without merging). Merge time represents the number of days between a PR's creation and merge date; we calculate this metric only for merged PRs. To assess statistical significance, we apply the Chi-square test to compare acceptance rates and the log-rank test to compare merge time distributions between the two groups ($\alpha = 0.01$).

Results. Regarding acceptance rates, we found that 91.9% of VRT-PRs (282/307 PRs) and 86.6% of Visual PRs (259/299) were merged. Although the acceptance rate of VRT-PRs was 5.3% points higher than that of Visual PRs, the difference was not statistically significant.

TABLE I: Resolution time (days) for merged PRs

Project	#PRs	Max	Mean	Median	Std
Visual-PRs	259	146.0	6.8	**1.2**	16.9
VRT-PRs	282	173.2	11.2	**4.5**	20.5

Table I summarizes the resolution times of VRT-PRs and Visual PRs. We found that the median resolution time for VRT-PRs is 3.8 times longer than that of Visual PRs. To assess the statistical significance of this difference, we conducted a log-rank test ($\alpha = 0.01$), which revealed a statistically significant difference in resolution times between the two groups. This diverges from Kuramoto *et al.* [28], who reported that visual issue reports did not significantly affect resolution time; VRT-PRs are instead resolved notably more slowly, suggesting VRT outputs trigger substantive review rather than routine confirmation.

Answer to RQ1. *The 5.3-point acceptance-rate difference was not significant, whereas VRT-PRs took significantly longer to be resolved.*

RQ_2: Are VRT-related Reviews Discussed More?

Motivation. Spadini *et al.* [31] observed that test files are nearly twice as likely to receive less discussion during code review when reviewed alongside production files. Since VRTs are a form of testing, they may exhibit similar patterns. However, VRTs differ from traditional test code (which focuses on functional aspects) by providing visual feedback on UI changes, potentially affecting how reviewers engage.

TABLE II: Effectiveness Measurement

Metric	Median		p-value	Effect Size
	Visual-PR	VRT-PR		
Commits	3	7	$<$ **0.001**	-0.348 (M)
Changed files	4	7	$<$ **0.001**	-0.285 (S)
Added lines	52	129.5	$<$ **0.001**	-0.266 (S)
Deleted lines	9	40.5	$<$ **0.001**	-0.302 (M)

Approach. We compare the number of comments between the merged VRT-PRs and Visual PRs (282 and 259 PRs, respectively). Specifically, we measure the total number of comments associated with each PR, including both review and discussion comments. To determine whether the differences are statistically significant, we apply the Mann-Whitney U test.

Additionally, we examine when comments containing links to VRT results appear in the review process to understand how VRT outputs are used during reviews. We normalize the comment index by the total number of comments in each PR to assess the relative timing of these VRT-related comments.

Results. We compared the number of comments in merged VRT-PRs and Visual PRs. The median number of comments in VRT-PRs was 10, 10.0 times as many as in Visual PRs (1 comment). This difference is statistically significant ($p < 0.01$), and the effect size (r = -0.809) indicates a large practical difference. This contrasts with two prior observations: Spadini *et al.* [31] found that test files receive less discussion than production code, and Kuramoto *et al.* [28] reported that visual information did not significantly increase discussion activity. VRT-PRs behave differently from both, eliciting an order of magnitude more comments.

Looking into the timing of comments that include links to VRT results, these links typically appear after 50.0% of the total comments have already been made (median). This suggests that VRT results are referenced not only during final validation but also throughout the discussion process, potentially facilitating earlier and more informed collaboration.

Answer to RQ2. *VRT-PRs receive significantly more comments than Visual PRs, indicating more active discussion. Additionally, VRT results are shared in the middle of the reviewing process rather than only at completion.*

RQ_3: Do VRT-related Reviews Require More Complex Fixes?

Motivation. Many studies [32][33] use metrics such as the number of commits and the size of code changes to characterize the complexity of fixing issues. Since VRTs often detect subtle visual differences across multiple components, they may require broader or more intricate modifications.

Approach. We examine whether VRT-PRs involve more extensive code changes compared to Visual PRs. We compare merged VRT-PRs (*i.e.,* 282 PRs) and merged Visual PRs (*i.e.,* 259 PRs) using metrics commonly used to characterize the scale and complexity of code modifications: number of commits, changed files, and lines added and deleted.

Results. Table II presents the distribution of code change metrics between Visual-PRs and VRT-PRs. The median number of commits in Visual-PRs is 3, compared to 7 in VRT-PRs. This

is 2.3 times larger. Other metrics also show more extensive changes in VRT-PRs. The median number of changed files, added lines, and deleted lines ranges from 1.75 to 4.5 times higher than in Visual-PRs. All differences are statistically significant ($p < 0.001$). The corresponding effect sizes indicate medium to small practical differences: r = -0.348 for commits (medium), -0.285 for changed files (small), -0.266 for added lines (small), and -0.302 for deleted lines (medium).

Answer to RQ3. *VRT-PRs exhibit about twice the median number of commits compared to Visual-PRs, along with 1.75 to 4.5 times more changed files and lines of code.*

RQ_4: What Problems Do Developers Discuss with VRT-Tests?

Motivation. Kuramoto *et al.* [30] categorized issues reported with images or videos, finding that visual evidence is frequently used to illustrate program behavior or UI layout. While VRTs similarly generate visual artifacts that developers reference during code review, the specific types of issues detected by VRTs remain unclear.

Approach. We used a card sorting approach [34] to classify the 344 issues found in merged VRT-PRs (*i.e.,* 344 of the 373 Chromatic-link comments). Two authors independently inspected review comments containing Chromatic links and examined the corresponding Chromatic outputs. They annotated the issues identified by VRTs to capture their intent, with each case allowed to receive multiple labels. The authors then discussed and reconciled their labels. The initial independent classification by the first and second authors achieved 71.8% label-level agreement, with disagreements on 118 of 419 labels. For disputed cases, a third author reviewed the conflicting labels and recommended resolutions. These recommendations were discussed with the original two authors until complete agreement was reached. The three authors have 8-17 years of programming experience.

It is worth noting that some PRs were written in non-English languages. Given the limited dataset size and the importance of maintaining diversity, we used translation services to interpret the content rather than excluding these PRs.

Results. During manual inspection, we initially created 32 labels and then grouped similar ones into 7 categories comprising 29 labels in total, as summarized in Table III. Of the 344 review issues inspected, we excluded 155 invalid cases: duplicate links (58 issues), inaccessible pages (17 issues), Chromatic errors (10 issues), new stories (61 issues), and those lacking sufficient context (9 issues), leaving 189 issues for analysis. The category distribution shows VRT's primary role: the three most frequent categories (Layout, Appearance, Color) all concern visual styling. However, 35 of the 189 issues (18.5%) involve non-stylistic origins: undefined component state (13, the State category), content disappearance (17: `Contents`, `Box`, and `Text Disappeared`), and visually imperceptible regressions (5, `No or Few Visual Differences Detected`). These cases suggest a secondary detection role.

Layout: This category accounted for 39.7% of issues (75/189) and was the most frequently observed type of visual regression. These VRT detections involved inconsistencies in the placement, spacing, and alignment of UI elements. The most prominent subtype was `Layout Shifted`, in which items were unintentionally repositioned, accounting for 42 cases. In one representative case, a developer identified a flaw in the baseline itself: an unintended widening of the left column. They argued that the updated rendering reflected the correct grid layout [35]. This detection prompted a broader discussion on layout principles, including the guideline that *"the grid should not break due to long section titles; instead, titles should wrap to multiple lines if necessary."* This case shows how VRT can reveal not only regressions but also misalignments in the baseline, triggering active discussion [36].

TABLE III: Categories of issues that developers discuss

Category	Labels
Layout (75)	Layout Shifted (42), Layout Changed (12), Size of Page/-Header Changed (11), Specific Items are Mis-aligned (10)
Appearance (52)	Box Size Changed (18), New Contents (10), Contents Disappeared (6), Box Shape Changed (6), Box Disappeared (4), Border Disappeared (3), Wrong Border Thickness (2), New Buttons (2), New Border Appeared (1)
Color (28)	Box Color Changed (16), Text Color Changed (8), Border Color Changed (4)
Text (18)	Text Disappeared (7), Contents of Text Changed (5), Text Size Changed (3), New Text (3)
State (13)	Undefined/Wrong State is Shown (8), Default State is Changed (5)
Test (12)	No or Few Visual Differences Detected (5), Fragile Snapshot (3), Focus Does Not Work (2), Flaky Test (1), No Changes (1)
Image (8)	Image Adjusted (6), Image Replaced (2)

Appearance: This category accounts for 27.5% of issues (52/189) and represents the second most frequently observed type of regression related to the visual appearance of the UI. Specifically, `Box Size Changed` (18 cases), `New Contents` (10 cases), and `Contents Disappeared` (6 cases) are the most common. In one case [37], when developers improved the basic layer structure of the UI, the impact extended beyond expectations, causing a panel to disappear from a separate component. VRT detected this change, and the discussions covered specific countermeasures, release timing adjustments, and a support policy for the older component.

Color: This category accounted for 14.8% of issues (28/189) and includes regressions related to the color scheme of UI elements, such as text color, background color, and line color. The most common subtypes were `Box Color Changed` (16 cases) and `Text Color Changed` (8 cases).

A representative example is shown in a PR [38] where VRT detected an unintended color change. The issue was originally caused by a previous PR that modified unrelated files and included no discussion of visual changes. Interestingly, the PR that introduced the regression affected different parts of the codebase and included no discussion of VRT results, highlighting two key insights: (1) visual regressions can propagate indirectly across seemingly unrelated changes, and (2) VRT is effective in surfacing such issues early, even when developers

are unaware of their visual impact.

Text: This category accounted for 9.5% of issues (18/189) and includes regressions related to text content and rendering. The most common subtypes were `Text Disappeared` (7 cases). In a representative case, a developer was notified by the VRT that a sentence had unexpectedly and unintentionally disappeared from the user interface [39].

State: This category accounted for 6.9% of issues (13/189) and includes regressions caused by incorrect or missing property settings or bugs in the UI. The most common subtype was `Undefined/Wrong State is Shown` (8 cases), where UI elements rendered unexpected content due to misconfigured properties. For example, developers identified a defect in which a variable was not correctly assigned, resulting in an undefined value being displayed in the UI [40].

Test: This category accounted for 6.3% of issues (12/189) and includes instances where no obvious visual differences were initially apparent, yet VRT still flagged changes. A representative sub-category is `No or Few Visual Differences Detected` (5 cases). In one such case, although VRT detected a visual difference, the developers were unable to perceive any noticeable change in the rendered UI [41].

Image: This category accounted for 4.2% of issues (8/189) and includes regressions related to images and pictures, with the most common subtypes being `Image Adjusted` (6 cases) and `Image Replaced` (2 cases). In one representative case, VRT detected visual discrepancies in an image component, where the aspect ratio had changed and the credit attribution was missing from the rendered output [42].

Answer to RQ4. *Of the seven categories identified, several (notably State, Test, and parts of Appearance and Text) consist of defects whose visual symptoms originate in functional, propagation-related, or sub-perceptual causes rather than stylistic ones, indicating that VRT functions as a detector of unintended consequences of code changes rather than a purely visual checker.*

V. Future Research Directions

This section discusses our findings and how they can inform the direction of future studies.

Distinguishing intentional changes from unintended regressions. RQ4 indicates that VRTs detect a diverse range of issue types that differ in nature. Additionally, RQ1 shows that issues involving VRT take longer to be fixed, and RQ2 revealed that VRT results are discussed during the review process. While these findings suggest that issues involving VRT are not trivial, they need evaluation based on whether they represent intentional changes or unintended regressions. For instance, most `Layout shifted` issues in the Layout category tend to occur unexpectedly, without developer intent. In contrast, `New Contents` in the Appearance category may generally reflect deliberate changes introduced by developers. Future work should identify whether each failure detected by VRT was expected or unintended and develop methods to make this determination automatically, thereby reducing investigation efforts.

Understanding the cause of visual regressions and their fixes. RQ3 reveals that the underlying code changes associated with VRT failures are typically more substantial, implying that larger or more complex modifications are more likely to introduce visual regressions. Future research should study the causes of unintentional test failures included in changes. Insights from these investigations could inform best practices for preventing visual regressions and support the development of automated repair techniques [43][44].

VI. Threats to Validity

We acknowledge several limitations that may affect the validity of our findings. Below, we discuss potential threats across three dimensions.

Threats to internal validity: This study relies on human annotations, which can be subjective. To mitigate the potential bias, we examined the results of VRTs independently and discussed conflicting annotations until full agreement.

Threats to construct validity: Our study reports associations but does not establish causality. VRTs may be disproportionately applied to PRs with substantial UI modifications, which inherently require more discussion and larger changes. The observed differences may partly reflect the complexity of underlying changes rather than the effect of VRT itself. Disentangling these effects would require matched-pair analyses comparing PRs of similar scope with and without VRT.

Threats to external validity: This study examines only 307 PRs that contain links to the results of VRTs. Although many projects today employ VRTs, it remains challenging to reliably identify both the test results and the associated discussions within PRs. To mitigate this issue, we included *all instances* we were able to retrieve at the time of data collection.

VII. Conclusions

This study analyzed 307 pull requests from GitHub repositories employing VRTs. Compared to Visual PRs, VRT-PRs show no significant acceptance-rate difference, but exhibit a median resolution time 3.8 times longer, 10 times more discussion comments (median 10 vs. 1), and 1.75 to 4.5 times larger code changes. These differences indicate that VRT-flagged issues are non-trivial and that VRT supports collaborative review; VRT results are typically shared around the midpoint of review, sustaining ongoing discussion rather than serving only as a final check.

Manual inspection of 189 issues yielded seven categories, with Layout (39.7%), Appearance (27.5%), and Color (14.8%) the most frequent. Several of these capture functional regressions such as undefined property values or disappearing components, indicating that VRT detection extends beyond stylistic verification.

Acknowledgment

We gratefully acknowledge the financial support of JSPS KAKENHI grants (JP24K02921, JP25K03100, JP26K23818), as well as JST BOOST (JPMJBS2423), ASPIRE grant (JPMJAP2415), and CREST grant (JPMJCR23M1, JPMJCR26X7).

References

[1] FSMdotCOM. (2020) 7 reasons why people uninstall apps. [Online]. Available: www.funkyspacemonkey.com/7-reasons-people-uninstall-apps

[2] T. Zhao, C. Chen, Y. Liu, and X. Zhu, "GUIGAN: learning to generate GUI designs using generative adversarial networks," in *Proceedings of the 43rd IEEE/ACM International Conference on Software Engineering (ICSE'21)*, 2021, pp. 748–760.

[3] Q. Chen, C. Chen, S. Hassan, Z. Xing, X. Xia, and A. E. Hassan, "How should i improve the ui of my app? a study of user reviews of popular apps in the google play," *ACM Transactions on Software Engineering and Methodology*, vol. 30, no. 3, 2021.

[4] B. J. Jansen, "The graphical user interface," *ACM SIGCHI Bulletin*, vol. 30, no. 2, pp. 22–26, 1998.

[5] S. R. Durgekar, S. A. Rahman, S. R. Naik, S. S. Kanchan, and G. Srinivasan, "A review paper on design and experience of mobile applications," *EAI Endorsed Transactions on Scalable Information Systems*, vol. 11, no. 3, 2024.

[6] E. Bertram, N. Hollender, S. Juhl, S. Loop, and M. Schrepp, "How to transfer user feedback into product improvements?" in *HCI (30)*, 2025, pp. 3–19.

[7] Z. Liu, C. Chen, J. Wang, Y. Huang, J. Hu, and Q. Wang, "Owl eyes: Spotting UI display issues via visual understanding," *CoRR*, vol. abs/2009.01417, 2020.

[8] ——, "Nighthawk: Fully automated localizing UI display issues via visual understanding," *IEEE Trans. Software Eng.*, vol. 49, no. 1, pp. 403–418, 2023.

[9] C. Bird, N. Nagappan, B. Murphy, H. C. Gall, and P. T. Devanbu, "Don't touch my code!: examining the effects of ownership on software quality," in *Proceedings of the 19th ACM SIGSOFT Symposium on the Foundations of Software Engineering (FSE'11)*, 2011, pp. 4–14.

[10] S. Baltes and V. Dashuber, "UX debt: Developers borrow while users pay," in *Proceedings of the 17th IEEE/ACM International Conference on Cooperative and Human Aspects of Software Engineering (CHASE'24)*, 2024, pp. 79–84.

[11] D. Nguyen. (2020) Chromatic2.0 - code review, but for ui. [Online]. Available: https://medium.com/@domyen/chromatic-2-0-code-review-but-for-ui-ddcebf0272b4

[12] S. Mahajan, K. B. Gadde, A. Pasala, and W. G. J. Halfond, "Detecting and localizing visual inconsistencies in web applications," in *Proceedings of the 23rd Asia-Pacific Software Engineering Conference (APSEC'16)*, 2016, pp. 361–364.

[13] I. Prazina, S. Becirovic, E. Cogo, and V. Okanovic, "Methods for automatic web page layout testing and analysis: A review," *IEEE Access*, vol. 11, pp. 13 948–13 964, 2023.

[14] Chromatic, "Why chromatic?" accessed: November 18th, 2025. [Online]. Available: https://www.chromatic.com/docs/

[15] ——, "Storybook," accessed: November 18th, 2025. [Online]. Available: https://www.chromatic.com/docs/#-storybook

[16] M. Watanabe, "What are developers actually discussing when visual regression tests fail?" 2026, created: May 16th, 2026. [Online]. Available: https://github.com/mmikuu/OnTheUseOfVisualRegressionTests

[17] "Format variations: Web: Display: Showcase, design: Picture, theme: Opinionpillar, mode: Light," 2025, accessed: November 18th, 2025. [Online]. Available: https://www.chromatic.com/test?appId=63e251470cfbe61776b0ef19&id=6570435efd2e01454c365e08

[18] "Process list uswds: Default," 2025, accessed: November 18th, 2025. [Online]. Available: https://www.chromatic.com/test?appId=65a6e2ed2314f7b8f98609d8&id=66047ec3c9f6fdbb985cc3b1

[19] Y. Lan, Y. Lu, Z. Li, M. Pan, W. Yang, T. Zhang, and X. Li, "Deeply reinforcing android GUI testing with deep reinforcement learning," in *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering (ICSE'24)*, 2024, pp. 71:1–71:13.

[20] A. Romano, Z. Song, S. Grandhi, W. Yang, and W. Wang, "An empirical analysis of ui-based flaky tests," in *Proceedings of the 43rd IEEE/ACM International Conference on Software Engineering (ICSE'21)*, 2021, pp. 1585–1597.

[21] Y. Lan, Y. Lu, M. Pan, and X. Li, "Navigating mobile testing evaluation: A comprehensive statistical analysis of android GUI testing metrics," in *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering (ASE'24)*, 2024, pp. 944–956.

[22] E. Alégroth, R. Feldt, and L. Ryrholm, "Visual GUI testing in practice: challenges, problems and limitations," *Empirical Software Engineering*, vol. 20, no. 3, pp. 694–744, 2015.

[23] E. Bui and H. Rocha, "Snapshot testing dataset," in *Proceedings of the 20th IEEE/ACM International Conference on Mining Software Repositories (MSR'23)*, 2023, pp. 558–562.

[24] S. Fujita, Y. Kashiwa, B. Lin, and H. Iida, "An empirical study on the use of snapshot testing," in *Proceedings of the 39th IEEE International Conference on Software Maintenance and Evolution (ICSME'23)*, 2023, pp. 335–340.

[25] V. P. G. Cruz, H. Rocha, and M. T. Valente, "Snapshot testing in practice: Benefits and drawbacks," *Journal of Systems and Software*, vol. 204, p. 111797, 2023.

[26] N. S. M. Yusop, J. Grundy, J. Schneider, and R. Vasa, "A revised open source usability defect classification taxonomy," *Information and Software Technology*, vol. 128, p. 106396, 2020.

[27] B. P. Robinson and P. A. Brooks, "An initial study of customer-reported GUI defects," in *Second International Conference on Software Testing Verification and Validation (ICST'09)*, 2009, pp. 267–274.

[28] H. Kuramoto, M. Kondo, Y. Kashiwa, Y. Ishimoto, K. Shindo, Y. Kamei, and N. Ubayashi, "Do visual issue reports help developers fix bugs?: a preliminary study of using videos and images to report issues on github," in *Proceedings of the 30th IEEE/ACM International Conference on Program Comprehension (ICPC'22)*, 2022, pp. 511–515.

[29] G. Docs. (2020) About the graphql api. [Online]. Available: https://docs.github.com/en/graphql/overview/about-the-graphql-api

[30] H. Kuramoto, D. Wang, M. Kondo, Y. Kashiwa, Y. Kamei, and N. Ubayashi, "Understanding the characteristics and the role of visual issue reports," *Empirical Software Engineering*, vol. 29, no. 4, p. 89, 2024.

[31] D. Spadini, M. F. Aniche, M. D. Storey, M. Bruntink, and A. Bacchelli, "When testing meets code review: why and how developers review tests," in *Proceedings of the 40th International Conference on Software Engineering (ICSE'18)*, 2018, pp. 677–687.

[32] H. Zhong and Z. Su, "An empirical study on real bug fixes," in *Proceedings of the 37th IEEE/ACM International Conference on Software Engineering (ICSE'15)*, 2015, pp. 913–923.

[33] A. E. Hassan, "Predicting faults using the complexity of code changes," in *ICSE*, 2009, pp. 78–88.

[34] D. Spencer and J. Garrett, *Card Sorting: Designing Usable Categories*. Rosenfeld Media, 2009.

[35] ioannakok, "Palettes: Event alt palette," accessed: November 18th, 2025. [Online]. Available: https://www.chromatic.com/test?appId=63e251470cfbe61776b0ef19&id=63e27ee5d6b8c3995850dc28

[36] ——, "Use css grid in section #7128," accessed: November 18th, 2025. [Online]. Available: https://github.com/guardian/dotcom-rendering/pull/7128#issuecomment-1422355751

[37] "Shell: Simple," 2025, accessed: November 18th, 2025. [Online]. Available: https://www.chromatic.com/test?appId=6266d45509d7eb004aa254fb&id=66ac13e797c819ac7db9e16d

[38] oliverlloyd, "bump source version #3897," accessed: November 18th, 2025. [Online]. Available: https://github.com/guardian/dotcom-rendering/pull/3897#issuecomment-1026674714

[39] sophie macmillan, "Remove decidepalette from layouts #9792," accessed: November 18th, 2025. [Online]. Available: https://github.com/guardian/dotcom-rendering/pull/9792#discussion_r1417022793

[40] Hiroshiba, "Add: Song's opening dialogue #2287," accessed: November 18th, 2025. [Online]. Available: https://github.com/VOICEVOX/voicevox/pull/2287#issuecomment-2408543634

[41] zachstence, "Clean up stories to fully enable chromatic (2/2) #2436," accessed: November 18th, 2025. [Online]. Available: https://github.com/evidence-dev/evidence/pull/2436#issuecomment-2318997943

[42] kasperbirch1, "Refactor media placeholder, making it a seperate component. ddfform-360 #542," accessed: November 18th, 2025. [Online]. Available: https://github.com/danskernesdigitalebibliotek/dpl-design-system/pull/542#issuecomment-1980864682

[43] T. Durieux, Y. Hamadi, and M. Monperrus, "Fully automated HTML and javascript rewriting for constructing a self-healing web proxy," in *Proceedings of the 29th IEEE International Symposium on Software Reliability Engineering (ISSRE'18)*, 2018, pp. 1–12.

[44] F. S. O. Jr., K. Pattabiraman, and A. Mesbah, "Vejovis: suggesting fixes for javascript faults," in *Proceedings of the 36th International Conference on Software Engineering (ICSE'14)*, 2014, pp. 837–847.